

Judgement Calls For Review

You don't need to open anything. Every decision below is reproduced in full, with the question attached. These are the places where a human judgement is baked into the code and I can only verify it is applied *consistently* — not that it is *right*.

Four findings on 2026-07-28 came from exactly this kind of reading, and none would have surfaced from code review: that a fixed pick must be exploitable, that setup moves need lookahead, that a team carrying Trick Room is not a Trick Room team, and that the open-sheet data is mostly an external archive.

Ranked by how much rides on the answer.

1. The playstyle classifier — highest stakes

`engine/playstyle.js`. This decides every team's label, and the matchup matrix, the Nash mixture and the retracted cycle are all built on top of it.

1a. The cascade order

```
function classify(six, sets) {
  if (hasMove(PERISH_MOVE) || (has(TRAP) && s.includes('gothitelle'))) return 'PerishTrap';
  if (hasMove(TR_MOVE) || (TR_prior(s) && slowLeaning(s))) return 'TrickRoom';
  if (moves.some(m => SETUP_MOVES.some(su => m.includes(su)))) return 'Setup';
  if (has(STALL) && s.filter(x => STALL.has(x)).length >= 2) return 'Stall';
  if (has(RAIN)) return 'Rain';
  if (has(SUN)) return 'Sun';
  if (has(SAND)) return 'Sand';
  if (has(SNOW)) return 'Snow';
  if ((hasMove(TW_MOVE) || has(TAILWIND)) && countAtk >= 3) return 'TailwindOffense';
  if (has(FAKEOUT)) return 'FakeOutBalance';
  return 'HyperOffense';
}
```

First match wins. Trick Room is checked **2nd of 11**, ahead of Setup, Stall and every weather.

So a Sun team carrying one Trick Room is labelled **Trick Room, not Sun**. That is very likely why Trick Room is the largest class at **26% of all teams**.

Question: what should the order be? Or should a team be allowed more than one label, with the matrix built over combinations? Note that Hyper Offense is the fallback — anything matching nothing lands there, which means "Hyper Offense" currently means "none of the above."

1b. The stall species list

```
const STALL = new Set(['amoonguss', 'dondozo', 'toxapex', 'alomomola',
  'blissey', 'clefairy', 'sinistcha']);
```

Seven species. A team needs **two or more** of them to be called Stall.

Question: right list, right threshold? *Sinistcha in particular also appears in your top archetypes as an offensive partner.*

1c. The setup moves

```
const SETUP_MOVES = ['dragon dance', 'swords dance', 'calm mind', 'nasty plot', 'bulk up',  
                    'quiver dance', 'shell smash', 'agility', 'iron defense', 'coil', 'work up'];
```

Question: is Agility setup? Is Iron Defense? *One boosting move makes the whole team "Setup", ahead of every weather in the cascade.*

1d. The slow-attacker list

```
function slowleaning(s) {  
  const SLOW_ATK = new Set(['torkoal', 'ursaluna', 'kingambit', 'incineroar',  
                           'basculegion', 'archaludon', 'crawdaunt', 'marowak']);  
  return s.some(x => SLOW_ATK.has(x));  
}
```

Used to call a team Trick Room even when Trick Room was never revealed — a Trick Room setter plus any one of these eight.

Question: right eight? *Kingambit and Incineroar are on most teams in the format, Trick Room or not.*

1e. The attacker count

```
if ((hasMove(TW_MOVE) || has(TAILWIND)) && countAtk >= 3) return 'TailwindOffense';
```

Question: why three?

1f. Classes too thin to be in a matrix

Out of 6,620 team-slots:

playstyle	slots	share	
Trick Room	1,740	26%	but see 1a
Sun	1,417	21%	
Setup	1,063	16%	
Tailwind Offense	797	12%	
Rain	625	9%	
Sand	287	4%	marginal
Hyper Offense	261	4%	marginal — and it's the fallback
Perish Trap	220	3%	marginal
Snow	134	2%	too thin
FakeOut Balance	65	1%	too thin
Stall	11	0.2%	unusable

The Nash solver currently assigns **5% of your strategy to Stall**, which has eleven observations. And cells like *"Tailwind Offense beats FakeOut Balance, 100% of 4 games"* are reported as decisive.

Question: drop the bottom three, or merge them into something? *Dropping is a judgement about what the format actually contains, not a statistical call.*

2. What counts as a usable game

`data/quality-filter.json`. This gate decides which games every model in the project ever sees. Currently **2,653 usable out of 20,387 collected — 13%**.

rule	current setting	question
named bots	username matches <code>pcrlbot</code> , <code>bot<digit></code> , <code><word>bot</code>	fine, but catches only self-identifying accounts
behavioural bots	50+ games AND exactly 1 distinct team	Is 50 right? Is "one team, ever" too strict for a dedicated human?
forfeits	excluded entirely	A forfeit records who quit, not who was winning — but it removes 1,551 games
minimum turns	3	Right floor?
require all four brought revealed	on — costs 487 games	This is the big one. See below

On the bring rule. It conditions on the game lasting long enough to reveal all four. Measured:

	games	mean turns
full bring (kept)	2,245	8.4
partial (dropped)	487	5.8

The kept set is systematically longer, and fast offensive Pokémon are over-represented among the games it throws away. The file already documents this limitation honestly — nobody had measured its size.

Question: is losing 487 games worth the guarantee? *The alternative is keeping them and treating the bring as partially observed.*

3. Move families — three entries do nothing

`engine/set_priors.js` . Decides which moves compete for one slot, so a generated set never carries both.

```
[ ['protect',    ['protect', 'detect', 'spikyshield', 'banefulbunker', 'burningbulwark',
                  'silktrap', 'obstruct', 'kingssshield']],
  ['tailwind',  ['tailwind']],
  ['weather',   ['raindance', 'sunnyday', 'sandstorm', 'snowscape', 'hail', 'chillyreception']],
  ['terrain',   ['electricterrain', 'grassyterrain', 'psychicterrain', 'mistyterrain']],
  ['trickroom', ['trickroom']],
  ['fakeout',   ['fakeout']],
  ].forEach(([fam, moves]) => { if (moves.length > 1) moves.forEach(m => { FAMILY_OF[m] = fam; }); });
```

Note the guard: `if (moves.length > 1)` . So `tailwind` , `trickroom` **and** `fakeout` **are never registered** — single-member families are skipped. They read as active and do nothing. Harmless today, misleading to the next reader.

Question: which pairs are missing? *The set sampler still generates two same-type attacking moves about 4 points more often than humans do (down from 10). Candidates that compete for a slot: two priority moves, two pivot moves (U-turn / Flip Turn / Parting Shot), two forms of recovery, two redirection moves.*

4. The 52 roles and the presence threshold

`engine/roles.py` . The first few roles:

```
speed_tailwind, speed_trickroom, speed_lower, weather_rain, weather_sun, weather_sand,
weather_snow, terrain_psychic, terrain_grassy, terrain_electric, terrain_misty,
abuser_psychic, abuser_grassy, abuser_electric, abuser_misty, fakeout, redirection, ...
```

```
PRESENT_AT = 0.50    # a TEAM counts as having a role when noisy-OR across its six reaches this
```

Question 1: are these the right roles at the right grain? *Four terrain setters and four terrain abusers is eight of the 52 spent on terrain, in a format where terrain barely appears in your archetypes.*

Question 2: is 0.50 the right bar? *And note it is applied to a number that overstates itself — the noisy-OR calculation assumes team members' roles are independent, and team building deliberately makes them not independent (you don't bring two Trick Room setters).*

5. Two hand-set numbers in WAR

engine/war.py :

```
MIN_GAMES = 30          # a species must appear on >= this many sixes to get a WAR
RIDGE = 6.0             # L2 strength – strong, because most species are rare (honest shrinkage)
```

Question: is 30 appearances enough to rate a species?

The ridge value is separately on the fix list — it needs choosing by a criterion rather than by feel, and engine/fit_policy.js already does that correctly twenty files away.

6. What MAG can see — the highest-value read

engine/board.js holds 47 features, each with a plain-English line. **This is how you found the Encore bug and the Prankster gate.** The most useful question is not whether any given feature is right, but **what mechanic is missing entirely.**

Known gaps, already confirmed:

- **No lookahead at all.** No rollouts, no search, no depth anywhere. MAG scores this turn and stops.
- **Volatiles are invisible.** Perish count, being Taunted, Encored, confused, or behind a Substitute. It knows "this move applies a volatile"; it cannot see one already on the field.
- **"Their defence is lowered" is dead** — essentially zero weight, range spanning zero. Across 82,836 human decisions the model could not detect it changes what people do.
- **No defensive type chart.** MAG sees what *it* hits for $\times 4$. It does not see that the thing in front of it hits *back* for $\times 2$.

Question: what else is missing that a player would notice immediately?

7. The one thing that needs a decision, not a review

Measuring MAG against a human. Every evaluation to date is against a random-clicking bot (93% of decisive pairs) or MAG's own previous version (55%). A random opponent shows the model is not broken; it says nothing about play against people.

The only human evaluation that exists is you playing it for ten minutes — which found **both** of that session's real bugs, making it the highest-yield testing in the project's history.

Two options, and the second is yours to rule on:

1. You play a set of games — 200 with the stopping rule agreed in advance.
 2. The bot ladders on the public server, which risks the account. I won't do that without your say-so.
-

How to answer

You don't need to write code. For any item above, "the order should be weather first", or "drop Stall and Snow", or "Agility isn't setup", or "30 is too low, make it 60" is enough — each is a one-line change once

the judgement is made.

The items where your answer changes the most downstream: **1a** (the cascade order, which propagates into every matchup number), **1f** (the thin classes, which are currently getting real Nash weight), and **6** (a missing mechanic, which is how the two genuine bugs were found).